Day37 作业：银行智能问答系统

本文可直接用 Typora 打开。项目使用虚构银行教学数据，不包含真实客户资料和任何密钥。2026-09-04验收范围：真实Elasticsearch、Redis、Neo4j + 确定性回答器 + LangChain Runnable适配；未接入真实大模型，不宣称完整复刻参考材料中的全部企业级RAG能力。

一、当天学习总结

1. 今天解决的核心问题

今天的主题不是“把三个数据库堆在一起”，而是让不同组件承担各自擅长的职责：

组件	主要职责	银行问答中的例子
Elasticsearch	全文搜索、相关度排序、过滤、高亮	搜索名称中含“稳健”的在售产品
Neo4j	查询客户、产品和风险之间的关系	查询 C001 的风险等级和可用金额
Redis	保存可过期的高频查询结果	第二次相同问题直接返回缓存
LangChain 思路	把输入、检索、业务规则和回答组织成处理链	让每一步的输入输出边界清晰

系统的主线是：

```
问题 + 客户编号
→ Neo4j读取最新客户画像
→ 分类为推荐、产品事实或领域外；领域外直接拒答
→ 生成包含画像指纹、产品目录指纹、问题摘要与数据版本的缓存键
→ Redis查询缓存
→ 未命中时由Elasticsearch与Neo4j检索候选
→ RRF融合两路名次
→ 事实问答定位明确产品；个性化推荐执行风险、金额、产品状态硬过滤
→ 生成带来源的回答
→ 写入有 TTL 的 Redis 缓存
```

2. 为什么需要混合架构

单一数据库无法同时做好全部工作：Elasticsearch 擅长文字，但不擅长多跳关系；Neo4j 擅长关系，但不能代替完整的全文相关度搜索；Redis 访问快，但不能充当长期事实来源。三者配合的价值来自“职责分工”，而不是组件数量。

3. 金融场景最重要的边界

相关度高不等于适合推荐。即使某个产品与用户问题非常相似，只要出现以下任一情况，也必须过滤：

- 客户可承受风险低于产品风险；
- 客户可用金额低于起投金额；
- 产品不是在售状态。

这些条件应该由确定性代码检查，不能只写进提示词并期待模型自觉遵守。

4. 缓存的正确理解

Redis 缓存的是“为了减少重复计算而保存的临时结果”。程序必须先读取最新画像，再把客户编号，姓名、风险等级、可用金额组成的画像指纹，产品目录指纹，规范化问题摘要和数据版本一起参与键生成。这样客户画像或产品目录变化后不会命中旧推荐；TTL只是第二层保护，不能代替正确的缓存合同。

5. 今天仍需继续掌握的内容

项目通过测试只说明工程在当前环境中满足合同，不等于我已经能够独立重写。我还需要完成：练习五闭卷查询体、一次数据库故障恢复、主链路口述和 20 道面试题随机抽问。

二、Elasticsearch 高频面试题 10 道

1. Elasticsearch 是什么？为什么银行问答系统要用它?

它是面向搜索和分析的分布式引擎。它建立倒排索引，能够进行全文匹配、BM25 相关度排序、结构化过滤和高亮。银行问答中可用于产品名称、产品说明、政策条款等检索。

2. 索引、文档、字段分别是什么?

索引类似一组同类数据的集合，文档是一条 JSON 记录，字段是文档中的键。例如 bank_products 是索引，一个理财产品是文档，name、risk_level 是字段。

3. text 与 keyword 有什么区别?

text 通常会分词，适合全文搜索；keyword 不分词，适合精确过滤、聚合和排序。产品描述应使用 text，固定品类或产品编号更适合 keyword。

4. match 与 term 有什么区别?

match 会分析查询文本，适合 text 全文匹配；term 直接匹配倒排索引中的精确词项，常用于 keyword。对品类精确过滤通常使用 term，对产品名称模糊搜索通常使用 match。

5. bool.must 与 bool.filter 有什么区别?

二者都能限制结果，但 must 中的查询通常参与相关度评分；filter 只判断是否满足条件，不计算相关度，并更容易缓存。关键词放 must，价格、品类等硬条件放 filter。

6. BM25 是什么?

BM25 是常见的词项相关度算法，会综合查询词在当前文档中的出现情况、文档长度和词在整个索引中的稀有程度。罕见且相关的词通常贡献更高分数。

7. Elasticsearch 高亮怎样工作?

在搜索请求的 highlight.fields 中声明字段，并配置 pre_tags、post_tags。命中后，服务端在 hit["highlight"] 中返回带标签片段；若没有高亮，代码应回退到 _source 原值。

8. 为什么价格区间适合放在 filter?

价格只决定“允许或不允许”，不需要让 5000 元比 4999 元具有更高文本相关度。放在 filter 语义更准确，也减少无意义评分。

9. 分片和副本是什么?

主分片把索引数据拆开，支持容量和查询横向扩展；副本是主分片的复制，提供故障恢复和读取能力。教学环境可以使用单节点、零副本，生产环境需要根据节点和可用性设计。

10. 搜索结果不正确时怎样排查?

先检查 mapping 与字段类型，再用 _analyze 查看分词，然后检查实际生成的查询体、索引数据、过滤条件和 _score。中文查询还要确认分词策略，不应只盯着 Python 打印结果。

三、Redis 高频面试题 10 道

1. Redis 是什么？本项目为什么使用它?

Redis 是以内存为主要存储介质的键值数据系统，访问延迟低。本项目用它缓存重复问答结果，减少重复的检索、业务过滤和答案组装。

2. Redis 常见数据类型有哪些?

包括 String、Hash、List、Set、Sorted Set、Stream 等。本项目的问答结果序列化后可放入 String；实际业务中的对象字段也可以使用 Hash。

3. TTL 是什么?

TTL 是键剩余的生存时间。键到期后会失效。缓存必须设置合理 TTL，否则旧产品信息可能长期返回，也可能让内存不断增长。

4. 什么是缓存穿透?

请求的数据根本不存在，导致每次都绕过缓存访问后端。可使用参数校验、短时缓存空结果、布隆过滤器等方式缓解，但空结果缓存时间不宜过长。

5. 什么是缓存击穿?

某个热点键到期时，大量请求同时访问后端。可使用互斥锁、逻辑过期、后台刷新或适当随机化过期时间处理。

6. 什么是缓存雪崩?

大量键在相近时间过期或 Redis 故障，造成请求集中冲击后端。可设置随机 TTL、多级缓存、限流、熔断和高可用架构。

7. 为什么本项目的缓存键必须包含客户编号和画像指纹?

回答受到客户风险和金额影响。若只按问题缓存，不同客户可能串用答案；若只有客户编号而没有画像指纹，同一客户风险等级或金额变化后也可能读到旧推荐。因此两部分都属于缓存合同。

8. Redis 持久化 RDB 与 AOF 有什么区别?

RDB 定期生成数据快照，文件紧凑、恢复快，但可能丢失最近一次快照后的数据；AOF 记录写命令，数据丢失窗口通常更小，但文件和恢复成本可能更高。纯缓存可以接受丢失时，持久化要求可低于核心账务系统。

9. 怎样实现原子地写值并设置过期时间?

使用一条带过期参数的 SET 命令，例如 SET key value EX 3600，而不是先 SET 再单独 EXPIRE，避免进程在两条命令中间失败留下永久键。

10. Redis 缓存是否能作为事实数据源?

通常不能。缓存可能过期、被淘汰或丢失。本项目的事实来源是 Elasticsearch 产品文档和 Neo4j 客户关系；Redis只是可重建的加速层。

四、练习五：复合查询与高亮显示

1. 题目合同

函数接收 keyword、min_price、max_price、category 四个可选参数：关键词进入 bool.must；价格范围和品类进入 bool.filter；按 _score 降序；最多返回 10 条；返回总命中数；名称关键词用 标签高亮。

2. 实现拆分

• exercise05/query_builder.py：只负责把参数转换成 Elasticsearch 查询体，容易做确定性测试。

• exercise05/search_products.py：负责调用真实 Elasticsearch，并把响应整理为结构化结果。

• exercise05/prepare_product_index.py：创建教学索引和固定产品数据。

• exercise05/run_exercise05.py：解析命令行参数并展示结果。

为了与前置练习二的数据合同一致，初始化脚本确定性生成100条完整商品文档，字段为product_id、name、category、price、stock、release_date。商品名使用空格，如phone model 001，使标准分析器能将phone独立分词；价格是float，因此命令行允许100.5一类边界。若索引已经存在但mapping、文档ID、数量、字段、值或类型不符，脚本应拒绝继续且保留现场，不能删除未知索引。

3. 关键理解

if min_price is not None 不能简单写成 if min_price，因为数值 0 是合法边界，但在布尔判断中会被当作假。hit.get("highlight", {}) 是为“没有高亮字段”提供空字典默认值；随后再回退到 _source["name"]，避免直接索引不存在的键而报错。

五、银行智能问答系统说明

1. 系统职责

系统接收客户编号和问题，先读取Neo4j最新客户画像并分类；天气等领域外问题立即拒答。其余问题构造带画像指纹、产品目录指纹和数据版本的Redis键。未命中时使用Elasticsearch与Neo4j检索并做RRF融合；事实问答定位明确产品并回答特点、风险等级或起投金额，个性化推荐则通过确定性代码执行风险、金额和状态过滤。

三库容器只绑定虚拟机127.0.0.1，Windows程序通过SSH本地端口转发访问。FastAPI的三个业务端点要求X-API-Key，真实密钥只保存在当前PowerShell的DAY37_API_KEY环境变量，不进入提交文件。

2. 关键模块

• models.py：定义客户、产品、搜索命中和问答结果的数据合同。

• business_rules.py：执行不可跳过的金融适配。

• cache.py：构建隔离缓存键并处理 TTL。

• elasticsearch_retriever.py：产品全文搜索适配器。

• prepare_bank_index.py：建立带中文ngram分析器的银行知识索引并写入固定产品。

• neo4j_repository.py：通过参数化 Cypher 读取画像和关系。

• rrf.py：用 Reciprocal Rank Fusion 合并多路名次。

• answer_generator.py：生成可复现的带来源回答。

• pipeline.py：按顺序组织整条问答流程。

• api.py：提供 FastAPI 接口。

3. 为什么保留离线版本

离线版本使用相同业务合同和固定教学数据，可以先验证代码逻辑，也能在数据库暂时不可用时学习。它不能冒充真实三库集成；真实结果必须由单独集成脚本验证并写入验收记录。

六、运行与提交说明

完整命令、预期结果和错误处理见 docs/01_从零复现操作手册.md。提交前必须运行测试和三库验证，并核对 docs/08_提交清单.md。提交包不得包含 .env、密码、Token、虚拟环境、数据库卷和缓存目录。

2026-09-04实际结果如下，详细命令与失败修复记录见 evidence/验收记录.md：

验收项目	实际结果
Python业务回归与静态检查	55项pytest通过；Ruff通过
VM身份保护	9项检查通过；明确IP与精确VMX的UUID匹配
三库与隧道	三个Day37容器健康；数据库只绑定VM回环地址
Elasticsearch	商品100条、银行知识5条；phone复合查询1条并返回高亮
Redis	写入、读回、正TTL通过；完整问答第二次命中缓存
Neo4j	3客户、5产品、4分类、5知识，共16条关系
完整问答链	教学客户C001只推荐P001，来源包含ES与Neo4j
FastAPI	TestClient连接真实三库；无/错密钥401、正确密钥200
问答评估	离线4/4、在线4/4；包含拒答、推荐与产品事实区分

练习五实际命中：商品17，名称 phone model 017，分类electronics，价格483.50元，库存18，总命中数1。

本次还修复了初始化保护遗漏：已有相同ID但内容或类型不符时，必须在写入前拒绝。新增8项测试先失败，再修复并完整回归，未放宽断言。

边界：没有运行真实LLM或嵌入模型，没有完成向量检索、生产负载/高可用验证；API验证不等于已启动对外HTTP服务。默认模板回答器保证本次结果可重复。Neo4j查询存在非阻断的弃用/缺失可选属性提示，后续可针对性清理。AI测试通过不等于学习者已掌握，也不保证授课教师按完整参考架构验收时没有补充要求。

七、个人学习声明

本次工程由 AI 辅助构建和验证。工程“能运行”不自动证明我能够独立实现。我将用 docs/07_独立复现训练.md 完成跟做、闭卷重写、故障恢复和口述复盘，并将真实证据与 AI 验收记录分开保存。